K L Deemed to be University

# Data Structures and Algorithms - 2

## Brand New Case Study Question Bank with Solution Keys

## CO1 - CO3 | Unique Topic Set

| | |
|---|---|
| **Student Name** | V.Shiva |
| **Roll No** | 2520090092 |
| **Course Code** | 25SC1305E |
| **Mode** | Project-Based Learning |
| **Coverage** | CO1, CO2, CO3 |
| **BTL Level** | 4 - Apply/Analyze |
| **Total Case Studies** | 3 complete case studies |
| **Marks Pattern** | 5 + 7 + 3 = 15 Marks each |

| CO No | Selected Topic | Case Study Theme | BTL |
|---|---|---|---|
| CO1 | AVL Tree Insert/Search | Blood bank priority unit indexing | 4 |
| CO2 | Segment Tree with Lazy Propagation | IoT lab temperature range minimum | 4 |
| CO3 | Kruskal MST with Union-Find | Rural water pipeline cost planning | 4 |

This PDF contains completely new case study scenarios compared with the earlier submissions. Each case study includes a realistic problem, structured data, diagram, sub-questions, solution key, and algorithm/pseudocode support for the code-related part.

# CASE STUDY 1 - CO1: Blood Bank Priority Unit Indexing using AVL Tree

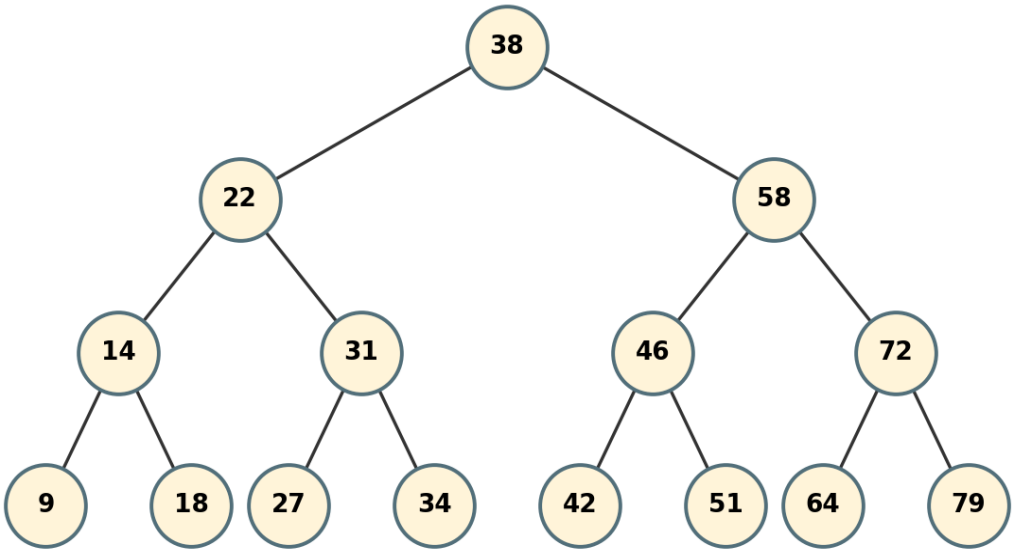
| | | | |
|---|---|---|---|
| **CO** | CO1 | **Topic** | AVL Tree Insert/Search |
| **Scenario** | Emergency blood unit priority-code index | **Marks** | 15 |
| **BTL** | 4 | **Main Skill** | AVL insertion, rotations, balance factors, O(log n) search |

## SCENARIO / CASE STUDY

A city blood bank stores emergency blood unit priority codes in an AVL Tree. Ambulance requests search the index by code, so the tree must stay balanced even when new units arrive in near-sorted priority order. The system records one batch of arrivals and verifies that lookups stay within predictable logarithmic height.

- Arrival sequence: 38, 22, 58, 14, 31, 46, 72, 9, 18, 27, 34, 42, 51, 64, 79.
- Search requests: 51 and 65. For a missing code, report predecessor and successor.
- AVL balance factor must remain -1, 0, or +1 after each insertion.

### Final AVL Tree for Blood Unit Priority Codes




| Sub | Question | Marks | CO/BTL |
|---|---|---|---|
| **(a)** | Construct the final AVL tree from the given priority-code insertion sequence. State the height in edges and the balance factor of the root. | 5 | CO1/BTL4 |
| **(b)** | Write AVL insert pseudocode with rotateLeft and rotateRight. Mention which rotation cases are checked after height update. | 7 | CO1/BTL4 |
| **(c)** | For search keys 51 and 65, write the search path. For missing key 65, state predecessor and successor and justify O(log n) lookup. | 3 | CO1/BTL4 |

## Solution Key - Case Study 1

Answer (a): The final AVL tree has root 38. The left subtree is rooted at 22 and the right subtree is rooted at 58. The height is 3 edges because the longest path such as 38 -> 22 -> 14 -> 9 or 38 -> 58 -> 72 -> 79 contains 3 edges. The root balance factor is 0 because left and right subtree heights are equal.

Answer (b): After every insertion, the algorithm updates node height, calculates balance factor, and checks LL, RR, LR, and RL cases. Single rotations fix LL/RR cases and double rotations fix LR/RL cases.

```
Node insert(Node node, int key) {
if (node == null) return new Node(key);
if (key < node.key) node.left = insert(node.left, key);
else if (key > node.key) node.right = insert(node.right, key);
```

```
else return node;

updateHeight(node);
int bf = height(node.left) - height(node.right);

if (bf > 1 && key < node.left.key) return rotateRight(node); // LL
if (bf < -1 && key > node.right.key) return rotateLeft(node); // RR
if (bf > 1 && key > node.left.key) { // LR
node.left = rotateLeft(node.left);
return rotateRight(node);
}
if (bf < -1 && key < node.right.key) { // RL
node.right = rotateRight(node.right);
return rotateLeft(node);
}
return node;
}
```

Answer (c): Search path for 51: 38 -> 58 -> 46 -> 51, found. Search path for 65: 38 -> 58 -> 72 -> 64 -> null/right, so 65 is missing. Predecessor is 64 and successor is 72. Since AVL height is maintained as O(log n), lookup remains efficient even as more blood units are inserted.

# CASE STUDY 2 - CO2: IoT Lab Temperature Monitoring using Segment Tree with Lazy Propagation

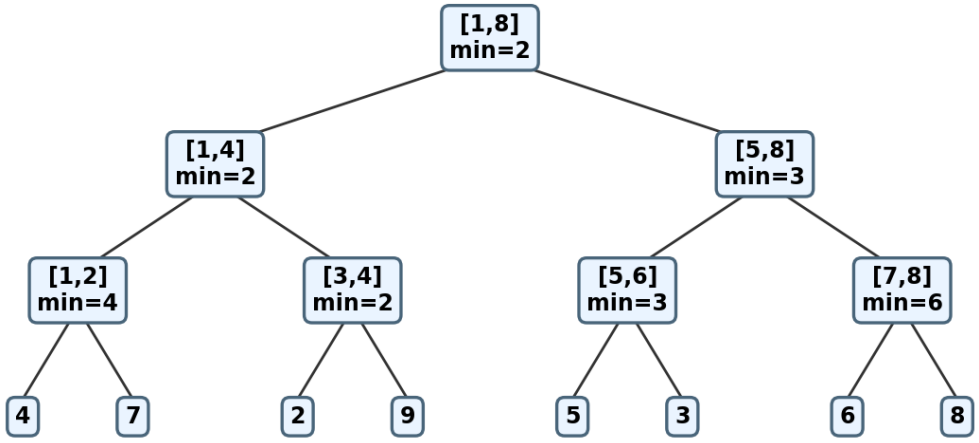
| | | | |
|---|---|---|---|
| **CO** | CO2 | **Topic** | Segment Tree with Lazy Propagation |
| **Scenario** | Range minimum query and range update | **Marks** | 15 |
| **BTL** | 4 | **Main Skill** | Build, range-min query, lazy range update |

## SCENARIO / CASE STUDY

An IoT lab monitors minimum temperature readings across eight sensor zones. The dashboard must answer range minimum queries quickly and also apply calibration changes to a full interval of sensors. A Segment Tree with lazy propagation is used so range updates do not require visiting every leaf immediately.

- Initial readings by zone 1 to 8: [4, 7, 2, 9, 5, 3, 6, 8].
- Query Q1: minimum from zone 3 to zone 7.
- Calibration update U1: add +2 to zones 4 to 6. Then query Q2: minimum from zone 3 to zone 7.

### Segment Tree for Lab Temperature Minimum Readings




| Sub | Question | Marks | CO/BTL |
|---|---|---|---|
| **(a)** | Construct the segment tree using minimum as the merge operation. Write root minimum and important node values for [1,4], [5,8], [3,4], and [5,6]. | 5 | CO2/BTL4 |
| **(b)** | Write pseudocode for rangeAdd(node,L,R,ql,qr,val) and queryMin(node,L,R,ql,qr) using lazy propagation. | 7 | CO2/BTL4 |
| **(c)** | Compute Q1 and Q2 after the update. Explain why lazy propagation is better than a normal array for frequent range updates. | 3 | CO2/BTL4 |

# Solution Key - Case Study 2

Answer (a): Root minimum for [1,8] is 2. Important node values are: [1,4] = 2, [5,8] = 3, [3,4] = 2, and [5,6] = 3. These are obtained by applying min(left child, right child) at each internal node.

Answer (b): Lazy propagation stores pending updates in a lazy array. When a segment is fully covered, the node value is updated immediately and the pending value is pushed to children only when required.

```
void push(int node) {
if (lazy[node] != 0) {
tree[2*node] += lazy[node];
lazy[2*node] += lazy[node];
tree[2*node+1] += lazy[node];
lazy[2*node+1] += lazy[node];
lazy[node] = 0;
}
}
```

```
void rangeAdd(int node, int L, int R, int ql, int qr, int val) {
if (qr < L || R < ql) return;
if (ql <= L && R <= qr) {
tree[node] += val;
lazy[node] += val;
return;
}
push(node);
int mid = (L + R) / 2;
rangeAdd(2*node, L, mid, ql, qr, val);
rangeAdd(2*node+1, mid+1, R, ql, qr, val);
tree[node] = min(tree[2*node], tree[2*node+1]);
}

int queryMin(int node, int L, int R, int ql, int qr) {
if (qr < L || R < ql) return INF;
if (ql <= L && R <= qr) return tree[node];
push(node);
int mid = (L + R) / 2;
return min(queryMin(2*node,L,mid,ql,qr),
queryMin(2*node+1,mid+1,R,ql,qr));
}
```

Answer (c): Q1 = min zones 3 to 7 = min(2, 9, 5, 3, 6) = 2. After adding +2 to zones 4 to 6, values become [4, 7, 2, 11, 7, 5, 6, 8]. Q2 = min(2, 11, 7, 5, 6) = 2. Lazy propagation is better because range updates and range queries both run in O(log n), while a normal array may need O(n) work for every range update or range query.

# CASE STUDY 3 - CO3: Rural Water Pipeline Planning using Kruskal MST

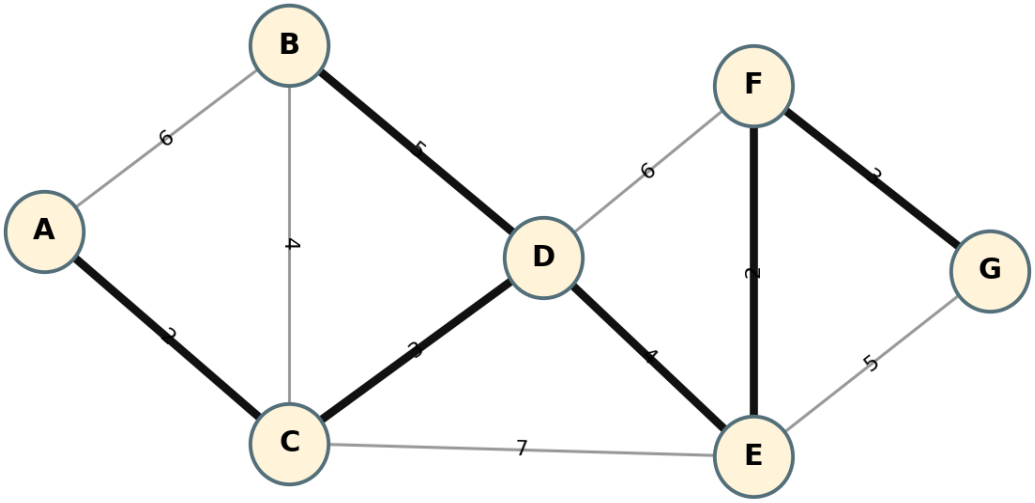
| | | | |
|---|---|---|---|
| **CO** | CO3 | **Topic** | Kruskal MST with Union-Find |
| **Scenario** | Minimum-cost water pipeline network | **Marks** | 15 |
| **BTL** | 4 | **Main Skill** | Sort edges, detect cycles, build MST |

## SCENARIO / CASE STUDY

A rural development office must connect seven villages through water pipelines with minimum total cost. The available routes form a weighted undirected graph. The selected pipeline network must connect every village without forming redundant cycles.

- Villages: A, B, C, D, E, F, G.
- Routes: A-B(6), A-C(2), B-C(4), B-D(5), C-D(3), C-E(7), D-E(4), D-F(6), E-F(2), E-G(5), F-G(3).
- Use Kruskal algorithm and Union-Find to select the MST.

**Rural Water Pipeline Network - Kruskal MST**




| Sub | Question | Marks | CO/BTL |
|---|---|---|---|
| **(a)** | Sort all routes in non-decreasing order of cost. Explain why the final spanning tree must have exactly 6 edges for 7 villages. | 5 | CO3/BTL4 |
| **(b)** | Apply Kruskal algorithm step by step using Union-Find. Mention accepted/rejected edges and write union-find pseudocode. | 7 | CO3/BTL4 |
| **(c)** | State the final MST cost. If route E-F(2) becomes unavailable, identify a valid replacement and explain the cost change. | 3 | CO3/BTL4 |

# Solution Key - Case Study 3

Answer (a): Sorted edges: A-C(2), E-F(2), C-D(3), F-G(3), B-C(4), D-E(4), B-D(5), E-G(5), A-B(6), D-F(6), C-E(7). A spanning tree on 7 vertices must contain n - 1 = 6 edges. Fewer edges cannot connect all villages and more edges would create a cycle.

Answer (b): Kruskal accepts the cheapest edge that connects two different components and rejects edges that form a cycle. Accepted edges can be A-C(2), E-F(2), C-D(3), F-G(3), B-C(4), and D-E(4). Once 6 edges are accepted, all 7 villages are connected.

```
int find(int x) {
if (parent[x] != x) parent[x] = find(parent[x]);
return parent[x];
}

boolean union(int a, int b) {
int ra = find(a), rb = find(b);
if (ra == rb) return false; // cycle
```

```
if (rank[ra] < rank[rb]) parent[ra] = rb;
else if (rank[ra] > rank[rb]) parent[rb] = ra;
else { parent[rb] = ra; rank[ra]++; }
return true;
}

int kruskal(List<Edge> edges) {
sort(edges by weight);
int cost = 0, count = 0;
for (Edge e : edges) {
if (union(e.u, e.v)) {
cost += e.w;
count++;
if (count == V - 1) break;
}
}
return cost;
}
```

Answer (c): Final MST cost = 2 + 2 + 3 + 3 + 4 + 4 = 18. If E-F(2) is unavailable, one valid replacement is E-G(5) or D-F(6) depending on the accepted components. Choosing E-G(5) gives new cost 21, increasing the cost by 3. The replacement must reconnect the separated component without creating a cycle.